

A Beginner's Guide to Real-Time Data Streaming

 systemdesignschool.io/fundamentals/introduction-to-kafka



Introduction to Kafka

Kafka 101: An Introduction to Kafka

Apache Kafka is a popular distributed streaming platform that enables real-time data processing and communication between various components in a system. Its architecture closely matches the general description of the log-based message queue described above. Kafka's core components include:

Event:

An event is a notification that something has happened. It contains the state of the system or an entity, encoded in formats like JSON, Avro, or Protobuf. In Kafka, events are stored as key-value pairs.

Topics:

Topics in Kafka are like tables in a database. They are used to categorize events and store them in an organized manner. Topics have partitions, which hold append-only logs. These logs are immutable and simple to reason about. Logs can only be read by offset, meaning that you must scan from the beginning; there is no random access. They are durable, with files stored on disks, and can be either permanent or temporary. Additionally, logs are easily replicable.

Partitions:

Partitions allow topics to be split across multiple nodes, increasing the scalability of the system. The assignment of a message to a partition is determined by the message key. If there is no key, the message is assigned using round-robin. If a key is present, a hash of the key decides the partition number. Keys preserve order, ensuring that messages with the same key are pushed to the same partition and maintain order due to the append-only nature of the logs. However, order is not guaranteed between partitions.

Brokers:

Brokers are the machines that run Kafka processes. They manage partitions, replications, and handle write and read requests.

Replication:

Partitions in Kafka are replicated to ensure durability and fault tolerance. This is achieved through leader-based replication, where one leader is responsible for writing data and managing N-1 followers. Replication is automatically managed and tunable by the producer for desired durability guarantees.

Producers and Consumers:

Developers primarily interact with Kafka through producers and consumers.

Producer:

The Producer and ProducerRecord APIs are used to write data to Kafka. Producers decide which partition to send data to based on the event's key.

Consumer:

The Consumer, ConsumerGroup, and ConsumerRecord APIs are used to read data from Kafka. A consumer group subscribes to a topic, and the partitions of that topic are divided among the consumers in the group. The number of consumers in a group should be less than or equal to the number of partitions.

Kafka Connect:

Kafka Connect is an integration framework that connects Kafka with other systems. It can act as a producer or a consumer. There is an ecosystem of existing connectors for systems like Elasticsearch or relational databases, allowing developers to avoid writing undifferentiated code. The Datagen connector, for example, serves as a useful demo data source.

Schema Registry:

The Schema Registry is an application separate from the broker that maintains a database (a Kafka topic) of schemas. The registry is cached for quick access. The consumer and producer APIs interact with the Schema Registry to ensure that messages are compatible with their respective schemas.

If you want to learn more about Kafka, check out Confluent's (developer of Kafka) Kafka 101 course: <https://developer.confluent.io/learn-kafka/apache-kafka/events/>, which has a hands-on part where you can try running Kafka on their infrastructure.